

Programmation Backend

Manipulation des fichiers



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Introduction



Contexte **général**

- Quand on parle de fichiers en PHP, on sous entend les fichiers **texte**. Ces fichiers peuvent contenir des données que l'on peut afficher sur une page Web ou peuvent servir d'espace de stockage pour y enregistrer le résultat de l'exécution de nos scripts PHP.
- Aussi, des fois, on a besoin de manipuler d'autres types de fichiers autres que les fichiers texte. Par exemple, le langage PHP a été conçu dès le départ pour fonctionner avec plusieurs bases de données, notamment MySQL. Mais tous les sites ne sont pas dynamiques et toutes les données ne proviennent pas de SGBD.
- De ce fait, PHP peut également exploiter des données de types **CSV** (Comma Separated Values, valeurs séparées par des virgules), fréquemment utilisés pour l'échange de données entre application (comme les tableurs par exemple).
- Les fichiers sont des entités dont on ne peut se passer. En effet, conserver des informations dans le but de les restituer, faire des statistiques, des fichiers d'accès, ... sont des éléments importants qu'on utilise constamment.



Fonctions de manipulation



Ces fonctions sont indépendantes du type de fichier. Elles s'appliquent à tous les fichiers manipulés en PHP.

- **Tests sur les fichiers:** Avant de manipuler un fichier, on vérifie son état.
- **Ouverture d'un fichier:** Tous les fichiers doivent être ouverts avant toute manipulation
- **Fermeture d'un fichier:** Toujours fermer un fichier après utilisation.
- **Gestion du système de fichiers:** Certaines fonctions manipulent les fichiers eux-mêmes.



Tests sur un fichier -1

- Avant de faire quoique ce soit sur un fichier; il faut souvent effectuer des tests pour connaître l'état de celui-ci (existe t'il, est t'il lisible ...).

file_exists

- Cette fonction permet de déterminer si un fichier existe et est accessible.
- Le fichier sous unix par exemple peut exister mais ne pas être accessible par l'utilisateur qui est propriétaire du démon httpd.
- Elle renvoie la valeur **true** si le fichier est présent

```
$fichier="donnees.dat";  
    if(file_exists($fichier)) {  
        print " Le fichier $fichier existe";  
    }else  
        print "Le fichier $fichier n'existe pas sur la machine ";
```



Tests sur un fichier -2

is_file et is_dir

- is_file permet de définir si l'élément sur lequel on travaille est un fichier.
- La fonction is_dir permet de tester si c'est un répertoire.
- Ces fonctions renvoient **true** si c'est un fichier ou si c'est un répertoire.

```
if(is_file("fichier.data"))  
    print "l'élément est un fichier";  
else  
    print "L'élément n'est pas un fichier";
```



Tests sur un fichier -3

filesize()

- Elle permet d'obtenir la taille en octets du fichier.

```
$taille=filesize($fichier);
```

unlink

- Cette fonction permet de détruire un fichier.

```
unlink($fichier);
```

copy

- Cette fonction permet d'effectuer une copie d'un fichier d'un endroit à un autre

```
copy($source_fichier,$destination_fichier);
```



Ouverture/Fermeture d'un fichier

Trois étapes à respecter :

- On ouvre le fichier avec **fopen**. Cette fonction renvoie une information qu'on doit mettre dans une variable (ici : \$monfichier). Cela nous sera utile pour fermer le fichier. On indique tout d'abord à **fopen** le fichier qu'on veut ouvrir (compteur.txt), puis comment on veut l'ouvrir (mode d'ouverture).
- On fait nos opérations de lecture/écriture sur le fichier.
- Enfin, quand on finit d'utiliser le fichier, on fait un **fclose** pour le fermer. On doit préciser quel fichier doit être fermé

Syntaxe

ressource fopen(nomdufichier, mode);

fclose(ressource retournée avec fopen)



Exemple d'exécution

```
<?php
// 1 : on ouvre le fichier
$monfichier = fopen('compteur.txt', 'r+');

// 2 : on fera ici nos opérations sur le fichier...

// 3 : quand on a fini de l'utiliser, on ferme le fichier
fclose($monfichier);
?>
```



Modes d'ouverture



Modes d'ouverture d'un fichier en PHP

Mode	Description
r	ouverture en lecture seulement
w	ouverture en écriture seulement (la fonction crée le fichier s'il n'existe pas)
a	ouverture en écriture seulement avec ajout du contenu à la fin du fichier (la fonction crée le fichier s'il n'existe pas)
r+	ouverture en lecture et écriture
w+	ouverture en lecture et écriture (la fonction crée le fichier s'il n'existe pas)
a+	ouverture en lecture et écriture avec ajout du contenu à la fin du fichier (la fonction crée le fichier s'il n'existe pas)

2

Gestion d'un fichier TXT



Lecture d'un fichier -1

Fonction file_get_contents()

- Elle permet de retourner une chaîne de caractères constituée du **contenu entier** du fichier \$fichier passé en paramètre.
- **Remarque: Cette fonction n'a pas besoin que le fichier soit ouvert avec fopen**

```
echo file_get_contents("document.txt");
```

Fonction fgetc():

- Elle sert à lire un seul caractère à la fois dans un fichier. C'est la méthode la plus fine de lecture d'un fichier.

```
echo fgetc($fichier);
```

- \$fichier : fichier ouvert avec fopen()
- la fonction retourne un caractère



Lecture d'un fichier -2

Fonction fgets()

- C'est la fonction correcte pour lire une ligne d'un fichier.
- Elle lit une ligne complète d'un fichier \$fichier ouvert avec fopen

```
echo fgets($fichier);
```

- On utilise souvent fgets() et fgetc() avec feof()
- La fonction feof() en PHP sert à détecter la fin d'un fichier lors de la lecture. Elle est très utilisée avec fgets() ou fgetc() pour lire un fichier dans une boucle.

```
$f = fopen("texte.txt", "r");  
while(!feof($f)){  
    echo fgets($f). "<br>";  
}  
fclose($f);
```



Exemple récapitulatif



Supposons qu'un fichier texte contient ceci:

ABC
DEF

Fonction	Lecture	fopen nécessaire	Résultat
fgetc()	caractère	oui	A B C D E F
fgets()	ligne	oui	ABC / DEF
file_get_contents()	fichier entier	non	ABC DEF
feof()	détecte fin du fichier	oui	



Ecriture dans un fichier -1

Fonction `file_put_contents($fichier,$contenu)`

- Elle permet d'intégrer le contenu `$contenu` dans le fichier `$fichier`.
- Pour que ce genre d'opérations s'exécute sans problème, il faut vérifier que les droits sur le fichier le permettent.

```
$str="Bonjour à tous";  
file_put_contents("contenu.txt",$str);
```

Fonction `fputs($fp,$chaine)`

- La fonction permet d'écrire dans le fichier la chaîne de caractère `$chaine` à partir de la position actuelle du pointeur.
- S'il y a déjà du contenu à l'endroit où l'on souhaite écrire, alors il sera écrasé lettre par lettre jusqu'à ce que l'écriture de `$chaine` finisse.



Ecriture dans un fichier -2

Fonction `fputc($fichier,$caractere)`

- Elle permet d'écrire un seul caractère dans un fichier.
- Elle est l'équivalent de `fgetc()` mais pour l'écriture.

```
$f=fopen("texte.txt","w");  
fputc($f,"P");  
fputc($f,"H");  
fputc($f,"P");  
fclose($f);
```

Fonction `fwrite()`

- Elle permet d'écrire une chaîne de caractères complète dans un fichier.
- La différence entre `fputs()` et `fwrite()` en PHP est en réalité quasi inexistante, car `fputs()` est simplement un alias de `fwrite()`. Cela signifie qu'elles utilisent la même fonction interne et produisent le même résultat.

3

Gestion d'un fichier CSV



Lecture d'un fichier

Fonction fgetcsv()

```
array fgetcsv ( resource $handle , int $length = 0 , string $delimiter , string $enclosure = '"', string $escape = '\\' )
```

- **Handle:** Un pointeur valide sur un fichier ouvert avec fopen(), popen() ou fsockopen().
- **Length:** Doit être plus grand que la plus grande ligne (en termes de caractères) à lire dans le fichier (y compris le caractère de fin de ligne). Ce paramètre est optionnel depuis PHP 5. Omettre ce paramètre (ou le définir à 0 en PHP 5.0.4 et suivant) fait que la longueur maximale de la ligne n'est pas limitée, ce qui est légèrement plus lent.
- **Delimiter:** Spécifie le séparateur (un seul caractère).
- **Enclosure:** Spécifie le caractère d'encadrement de texte (un seul caractère).
- **Escape:** Définit le caractère d'échappement (un seul caractère). Par défaut, c'est un antislash



Exemple

```
<?php
    $fichier = fopen("Classeur1.csv", "r");
    while(!feof($fichier)){
        $ligne = fgetcsv($fichier,0,";");
        echo $ligne[0]."<br>".$ligne[1];
    }
    fclose($fichier);
?>
```



```
<?php
    $fichier = fopen("Classeur1.csv", "r");
    while(($ligne = fgetcsv($fichier,0,";")) == true){;
        echo $ligne[0]."<br>".$ligne[1];
    }
    fclose($fichier);
?>
```

L'utilisation de la fonction feof dans la condition d'arrêt engendre une erreur. C'est un comportement classique en PHP lorsque l'on lit un fichier ligne par ligne. En fait, feof() devient true seulement après avoir essayé de lire la fin du fichier



Ecriture dans un fichier

Fonction fputcsv()

```
int fputcsv ( resource $handle , array $fields , string $delimiter = ',' )
```

- **Handle:** Le pointeur de fichier doit être valide et pointer sur un fichier ouvert avec succès par fopen() ou fsockopen() (et pas encore fermé par fclose()).
- **Fields:** Un tableau de valeurs.
- **Delimiter:** paramètre optionnel qui spécifie le délimiteur (un seul caractère).



Exemple

```
<?php
    $fichier = fopen("Classeur1.csv", "a+");
    $tab[]="aaaa";
    $tab[]="bbbb";
    $tab[]=1;
    $ligne = fputcsv($fichier,$tab,";");
    echo "ok";
    fclose($fichier);
?>
```